

TOKYO, JAPAN / DECEMBER 11-13, 2025

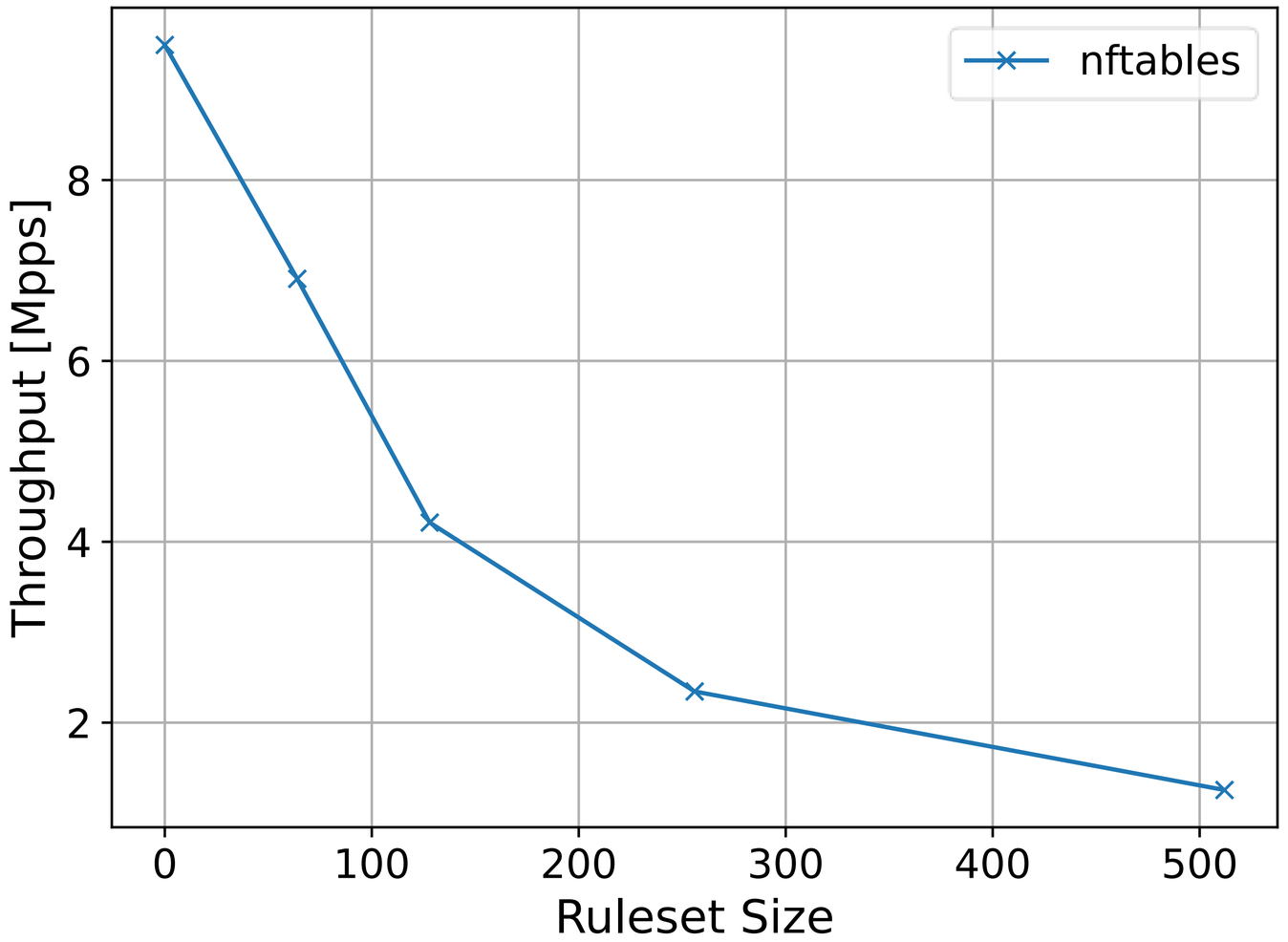
Locality-Aware, Self-Adjusting Data Structures: A Case Study in Linux Firewalls

Jonas Köppeler, Maciej Pacut, Stefan Schmid – *TU Berlin*
Vamsi Addanki – *Perdue University*
Tamás Lévai, Gábor Rétvári - *Budapest University of Technology and Economic*



東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Packet classification is expensive.



nftables

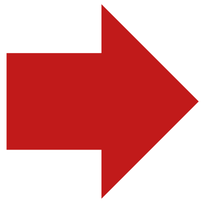
```
table inet my_table {

  set trusted_ports {
        type inet_service
        elements = { 80, 443, 22 }
  }

  chain input {
    type filter hook input priority 0;
      ip saddr 127.0.0.1 accept
      ip daddr 198.168.0.0/16 drop
      tcp dport @trusted_ports accept
  }
}
```

nftables Virtual Machine

```
ip saddr 127.0.0.1 accept
ip daddr 198.168.0.0/16 drop
```



```
payload load 4b @ network
   header + 12 => reg 1
cmp eq reg 1 0x0100007f
immediate reg 0 accept

payload load 2b @ network
   header + 16 => reg 1
cmp eq reg 1 0x0000a8c6
immediate reg 0 drop
```

Speedup Strategies

- Rule reductions/optimizations
- Ruleset-structure-exploiting algorithms
- Hardware offloading
- **Our approach:** Exploit locality in network traffic

Move Recursively Forward (MRF)

- **Goal**: Dynamically push **heavy-hitter rules** towards the front

Move Recursively Forward (MRF)

- **Goal**: Dynamically push **heavy-hitter rules** towards the front
- **Challenge:** Rule reordering must **preserve firewall logic**

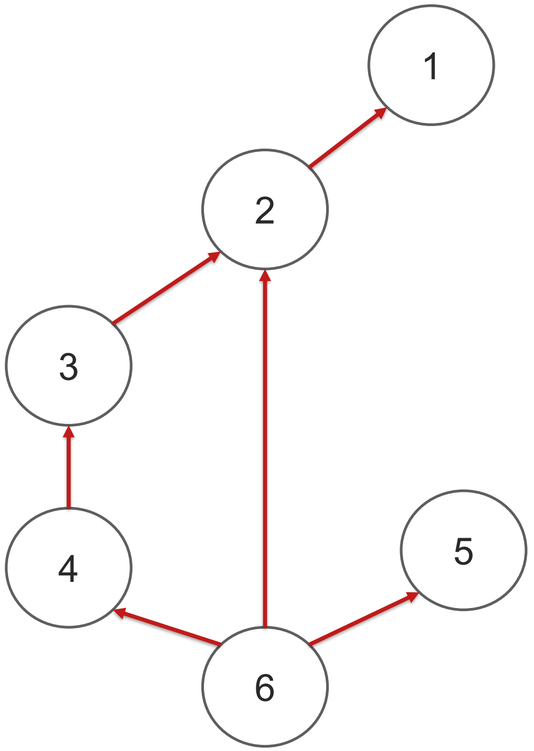
Move Recursively Forward (MRF)

- **Goal**: Dynamically push **heavy-hitter rules** towards the front
- **Challenge:** Rule reordering must **preserve firewall logic**
- **MRF**: Safely moves rules **as far forward as logic allows**

Move Recursively Forward (MRF)

- **Goal**: Dynamically push **heavy-hitter rules** towards the front
- **Challenge:** Rule reordering must **preserve firewall logic**
- **MRF**: Safely moves rules **as far forward as logic allows**
- Simulation results show:
  - Ruleset *small* and *high locality* of reference: 14x improvement
  - Otherwise: similar performance to static list

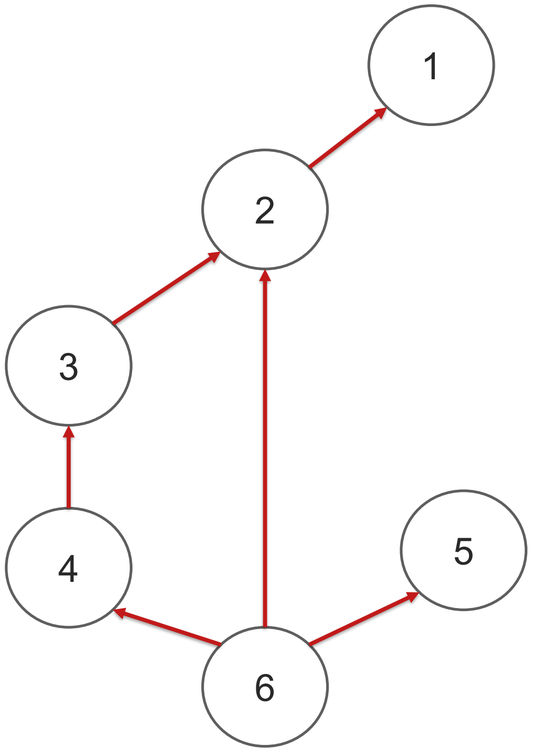
MRF Example



N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

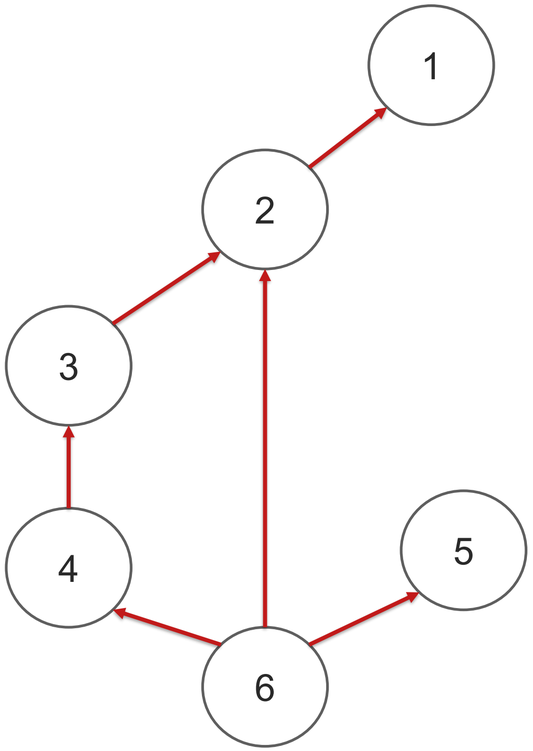
Incoming packets: P5, P1,P3



N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

Incoming packets: P1,P3

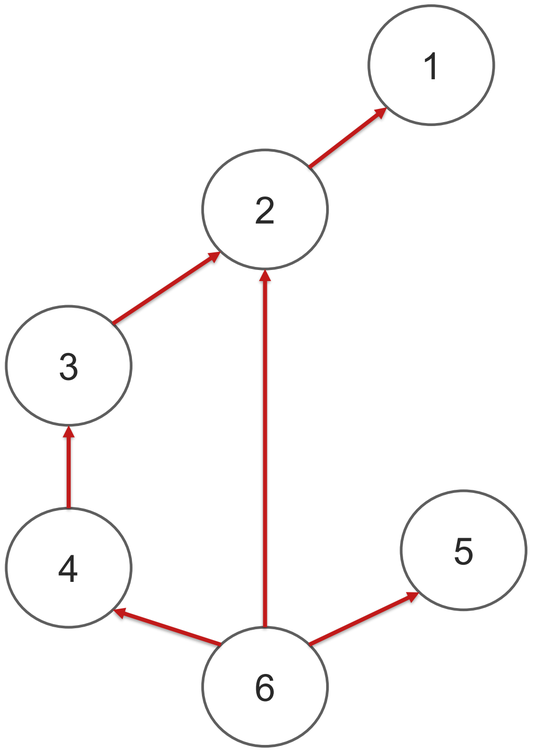


P5

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

Incoming packets: P1,P3

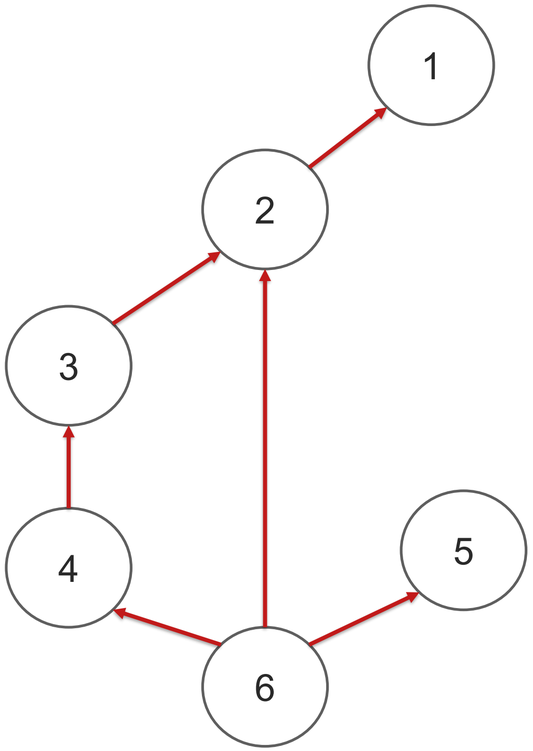


P5

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

Incoming packets: P1,P3

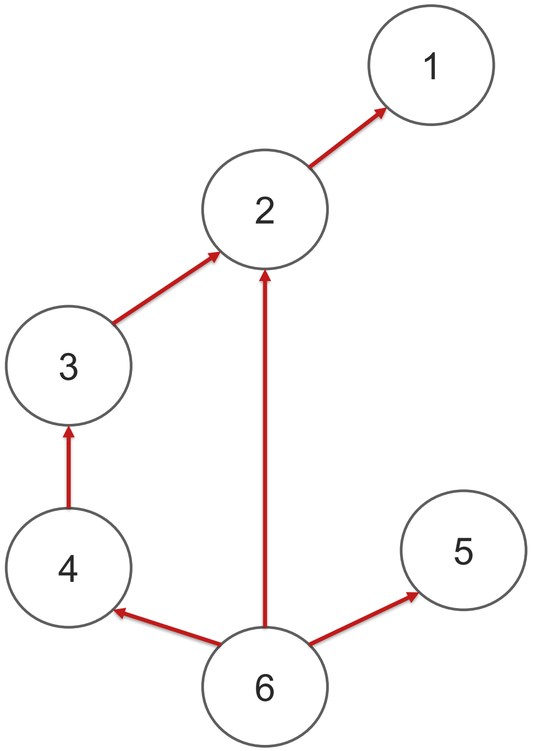


P5

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

Incoming packets: P1,P3

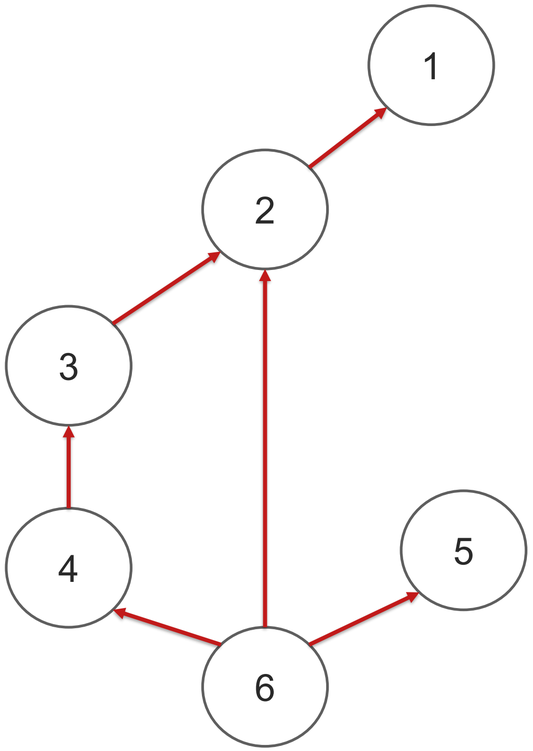


P5

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

Incoming packets: P1,P3

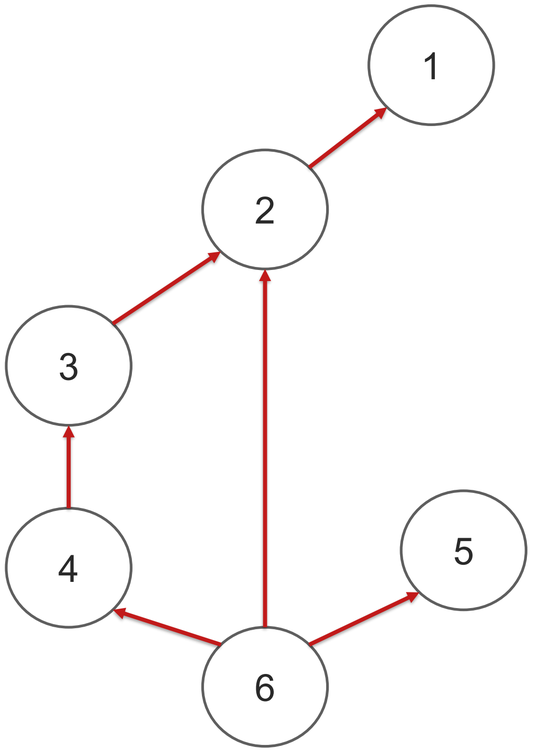


P5

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

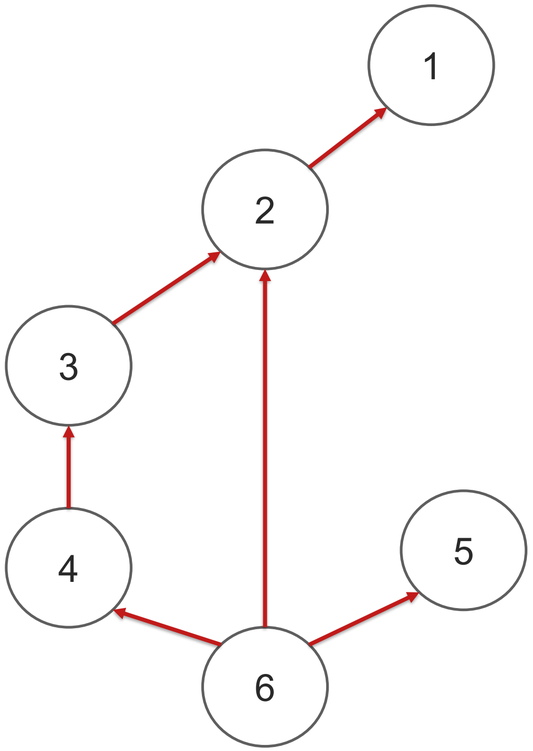
Incoming packets: P3



P1

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

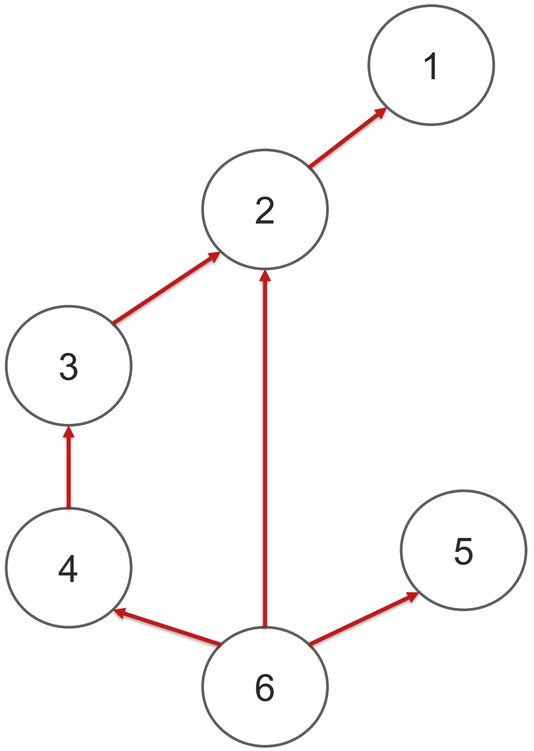


Incoming packets: P3

P1

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

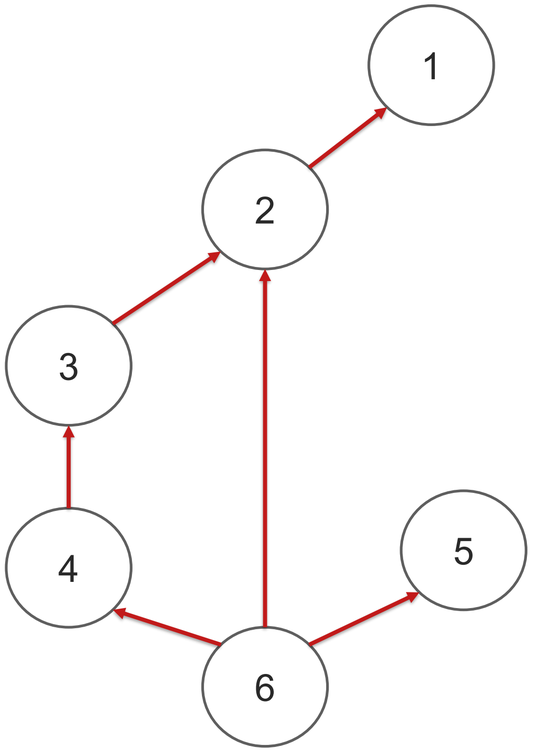


Incoming packets:

P3

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

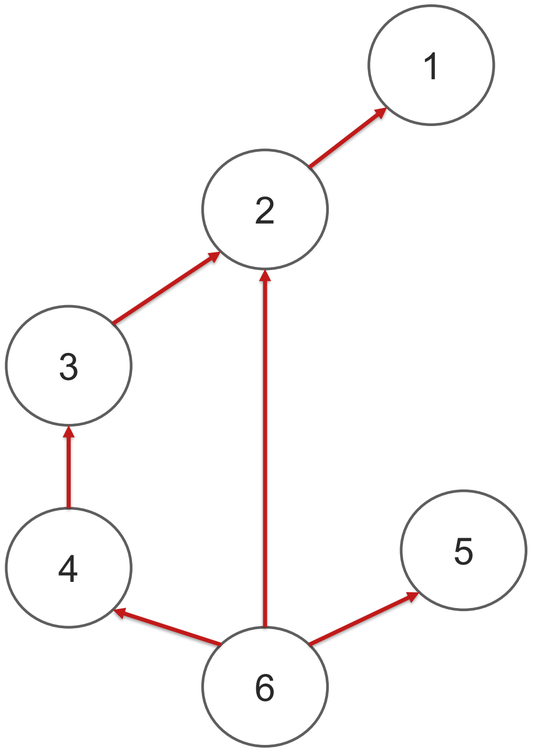


Incoming packets:

P3

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example

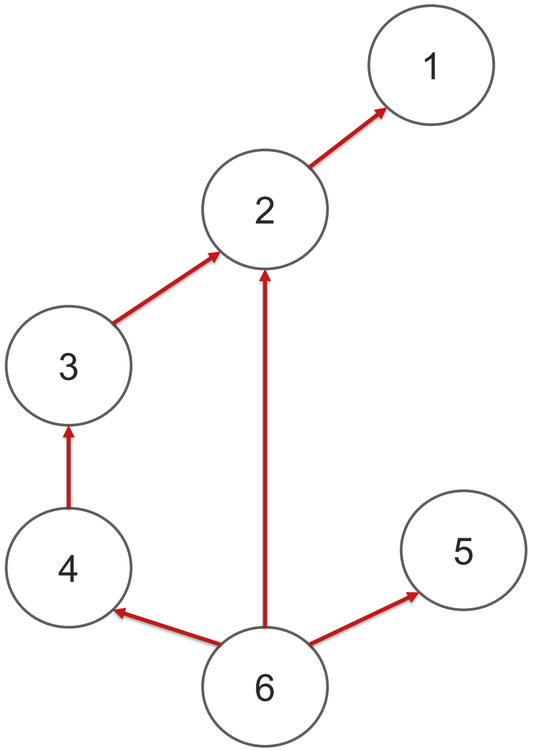


Incoming packets:

P3

N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

MRF Example



Incoming packets:

P3

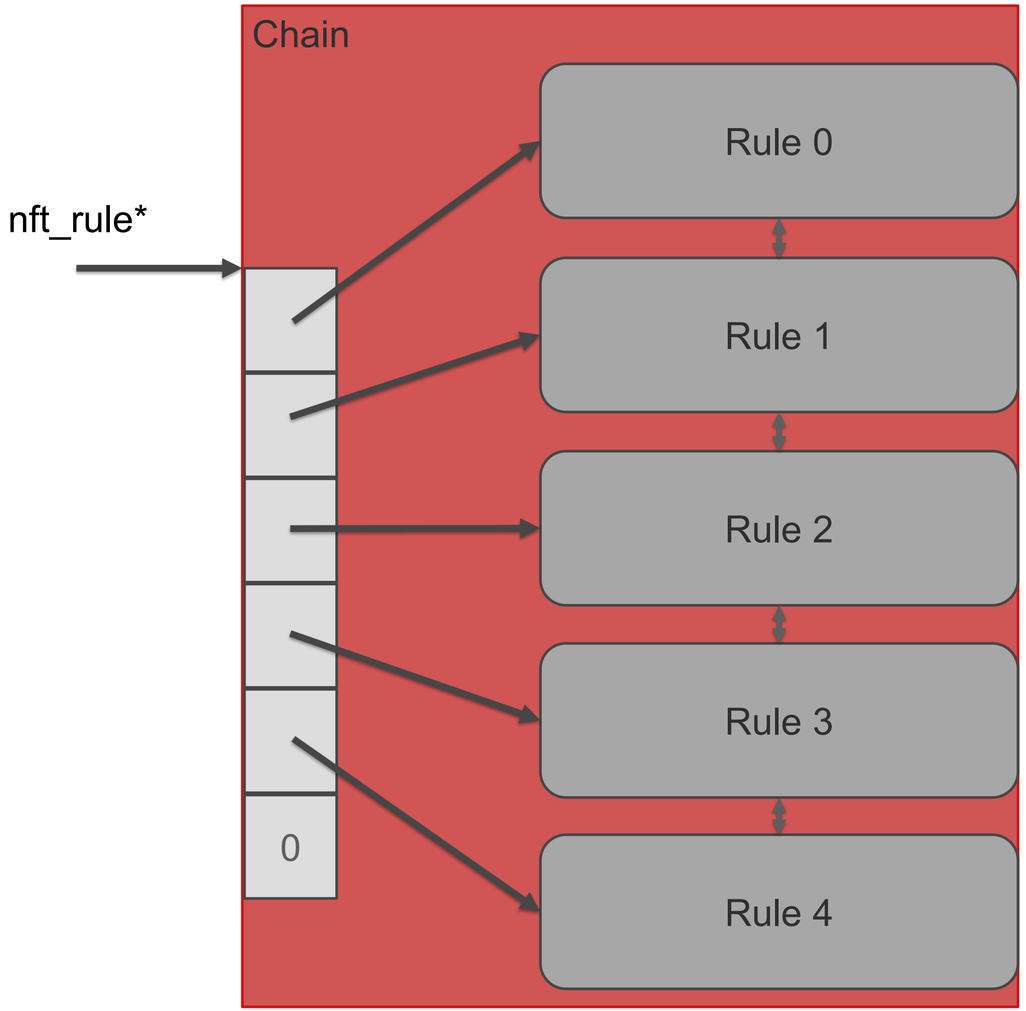
N	Proto	Src IP	Dst IP	Src Port	Dst Port	Action
1	TCP	10.12.12.0/24	20.0.0.1/32	*	80	DROP
2	TCP	0.0.0.0/0	20.0.0.1/32	*	80	ACCEPT
5	UDP	20.0.0.0/24	10.0.10.0/24	*	3306	ACCEPT
3	*	0.0.0.0/0	20.0.0.1/32	*	*	DROP
4	UDP	0.0.0.0/0	0.0.0.0/0	1000-2000	1000-2000	ACCEPT
6	*	0.0.0.0/0	0.0.0.0/0	*	*	DROP

Performance Considerations

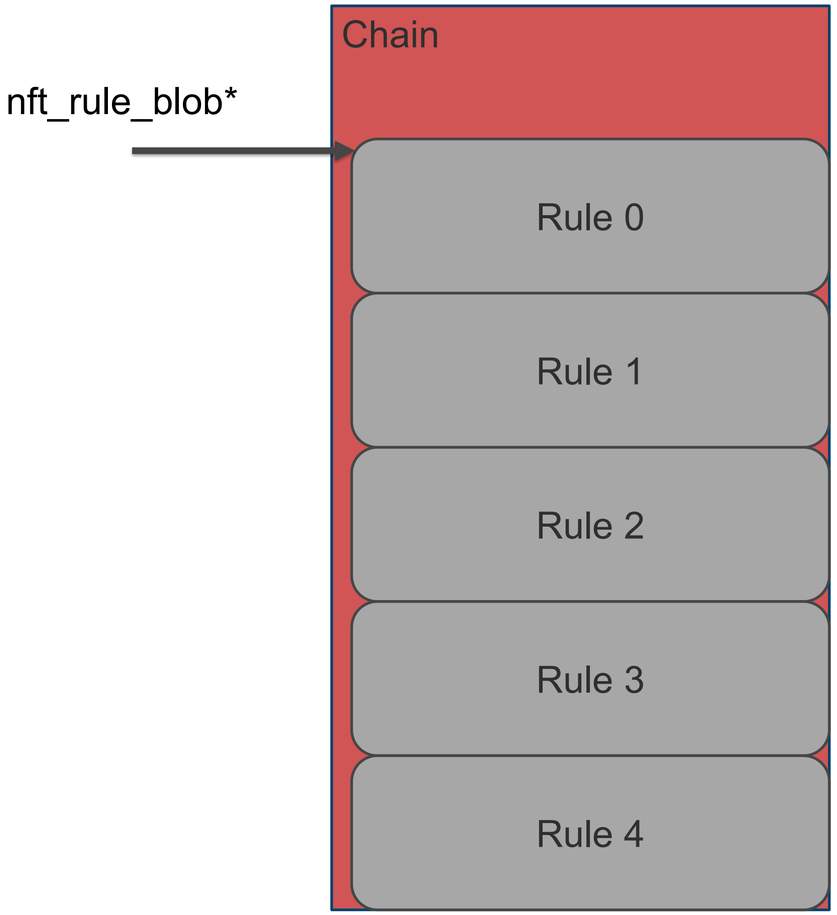
- **Locality** of reference
  - Depends on *traffic*
- Number of **dependencies** / freedom of rearrangement
  - Depends on *ruleset*
- Cost of **reordering**
  - Depends on *implementation*
- Cost of **dependency detection**
  - Depends on *implementation*

nftables Versions

Version < 5.17

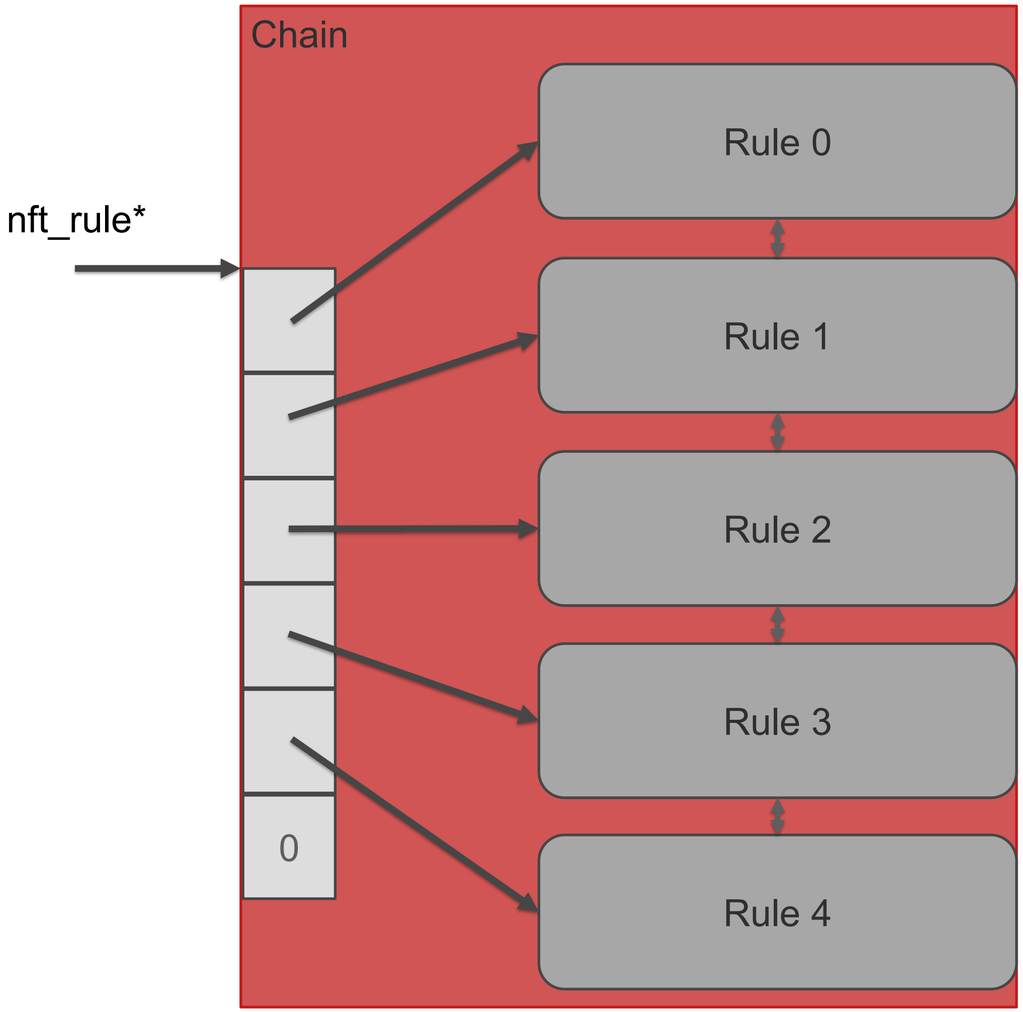


Version >= 5.17



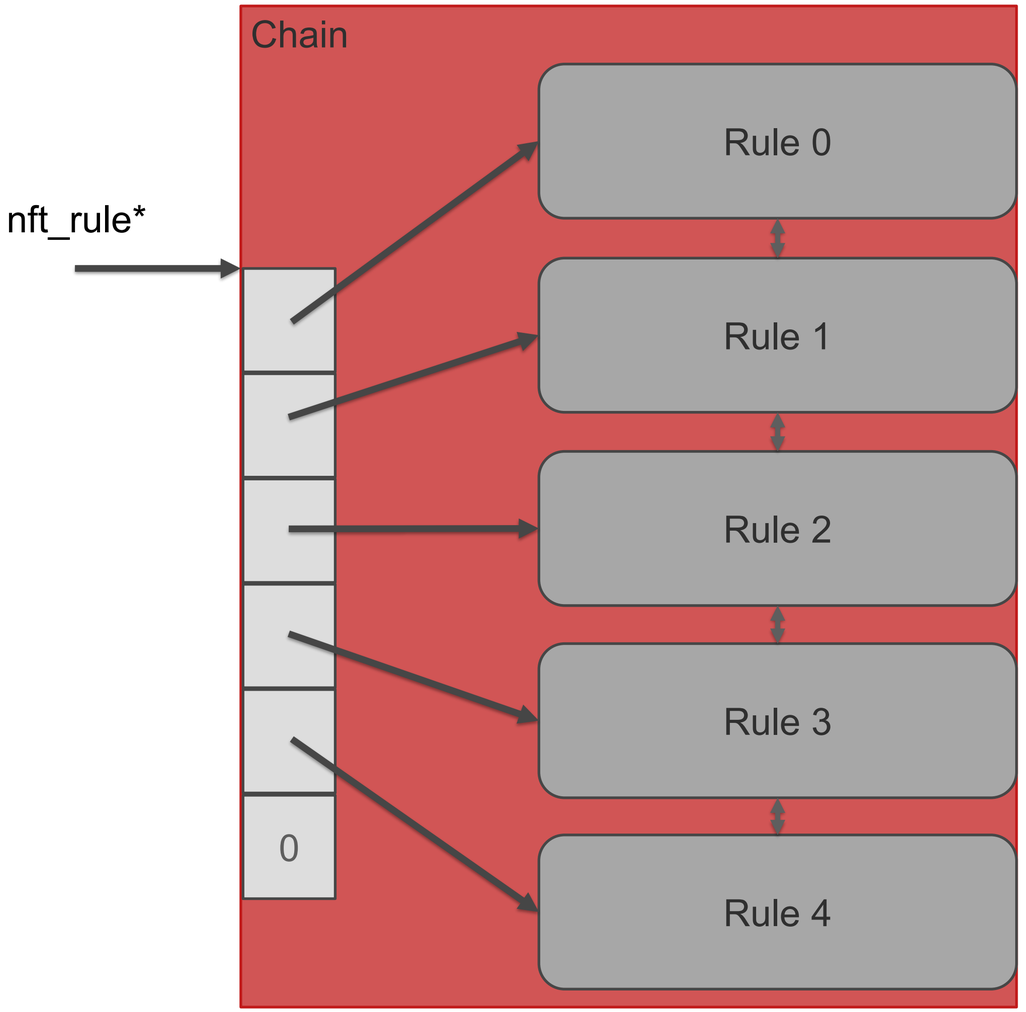
Implementation: Single Core

- Prototype implementation based on pointer array
  - Reduces the cost of swaps at expense of more memory
- Focus on 5 tuple (proto, srcIP, dstIP, srcPort, dstPort)



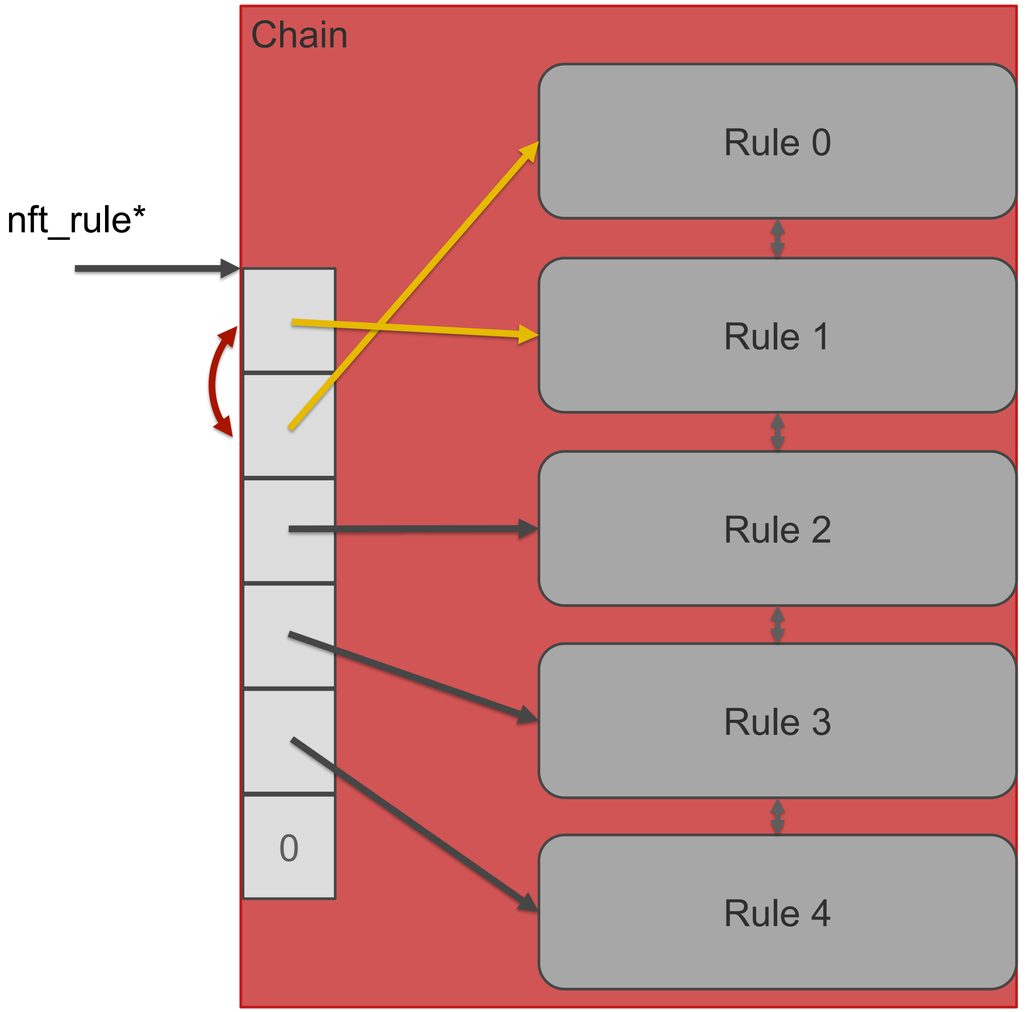
Implementation: Single Core

- Dependency detection:
  Extract match-ranges from bytecode
- Store range-helper struct along with rule
- During reorder: Check if ranges overlap



Implementation: Single Core

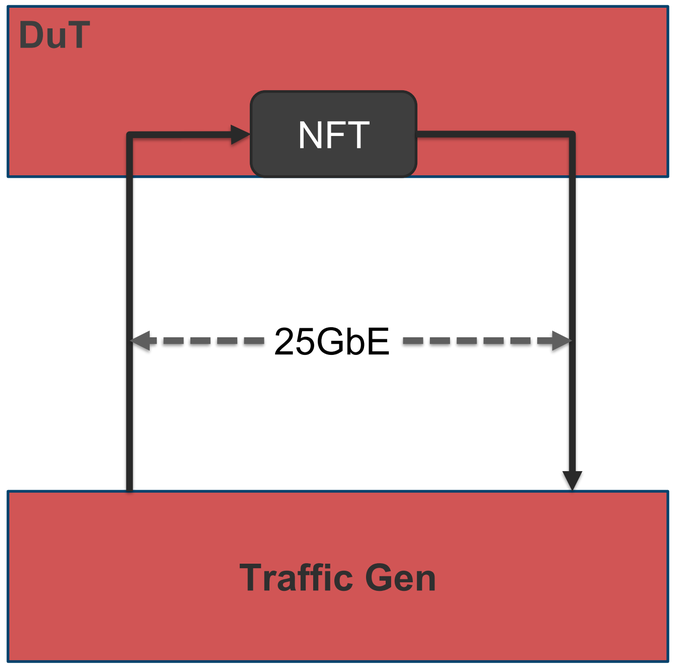
- Reorder rules
  - Swap pointers
  - Leave linked list untouched
- Rule insertions
  - May insert new dependencies
  - Reset pointer array



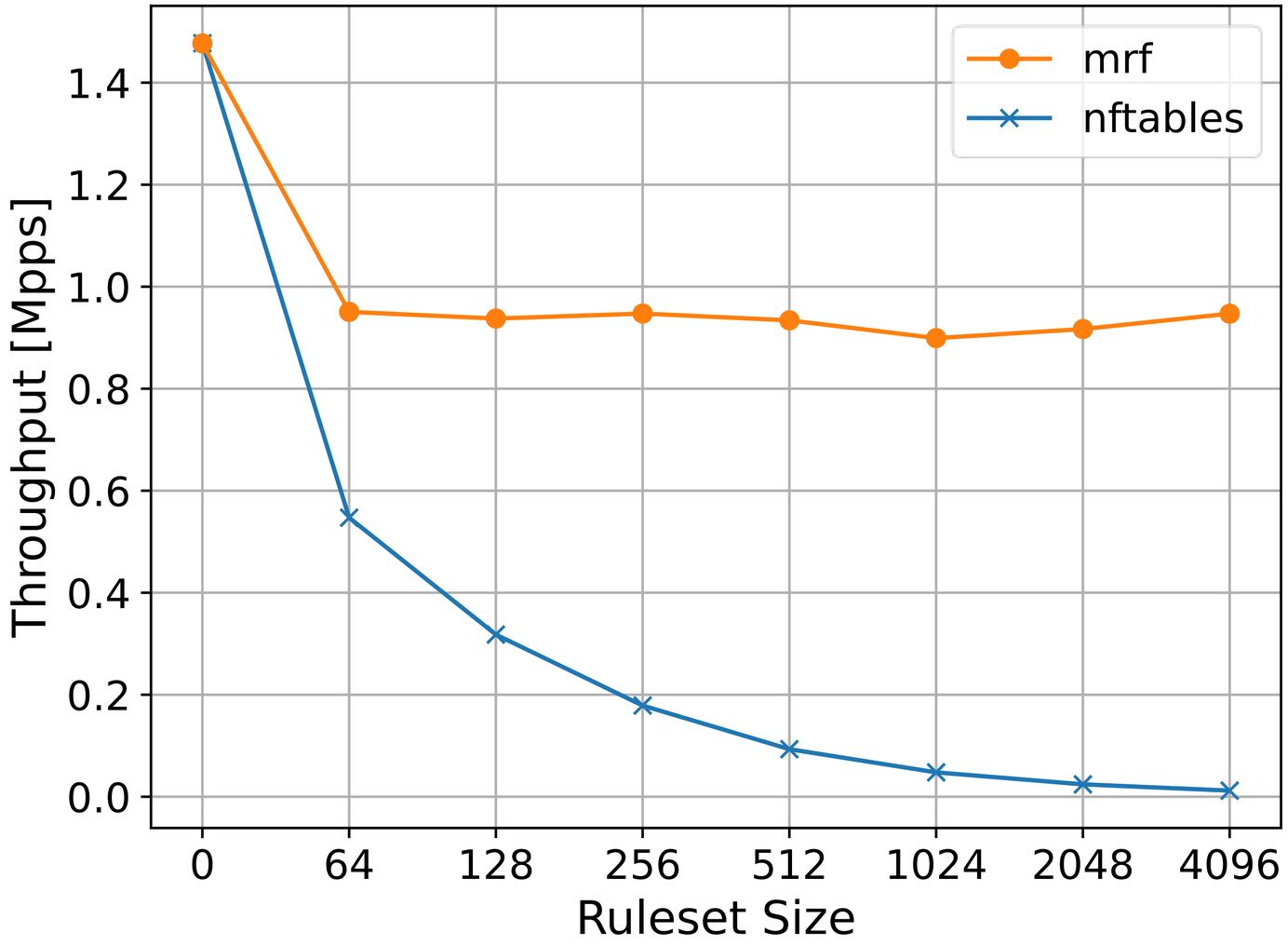
Evaluation: Setup

- Rulesets:
  - All targets set to *Accept*
  - Variable number of dependencies
- Traffic:
  - Contains X flows
  - Flows always match the last X rules
  - Repeated X-packet cycle:

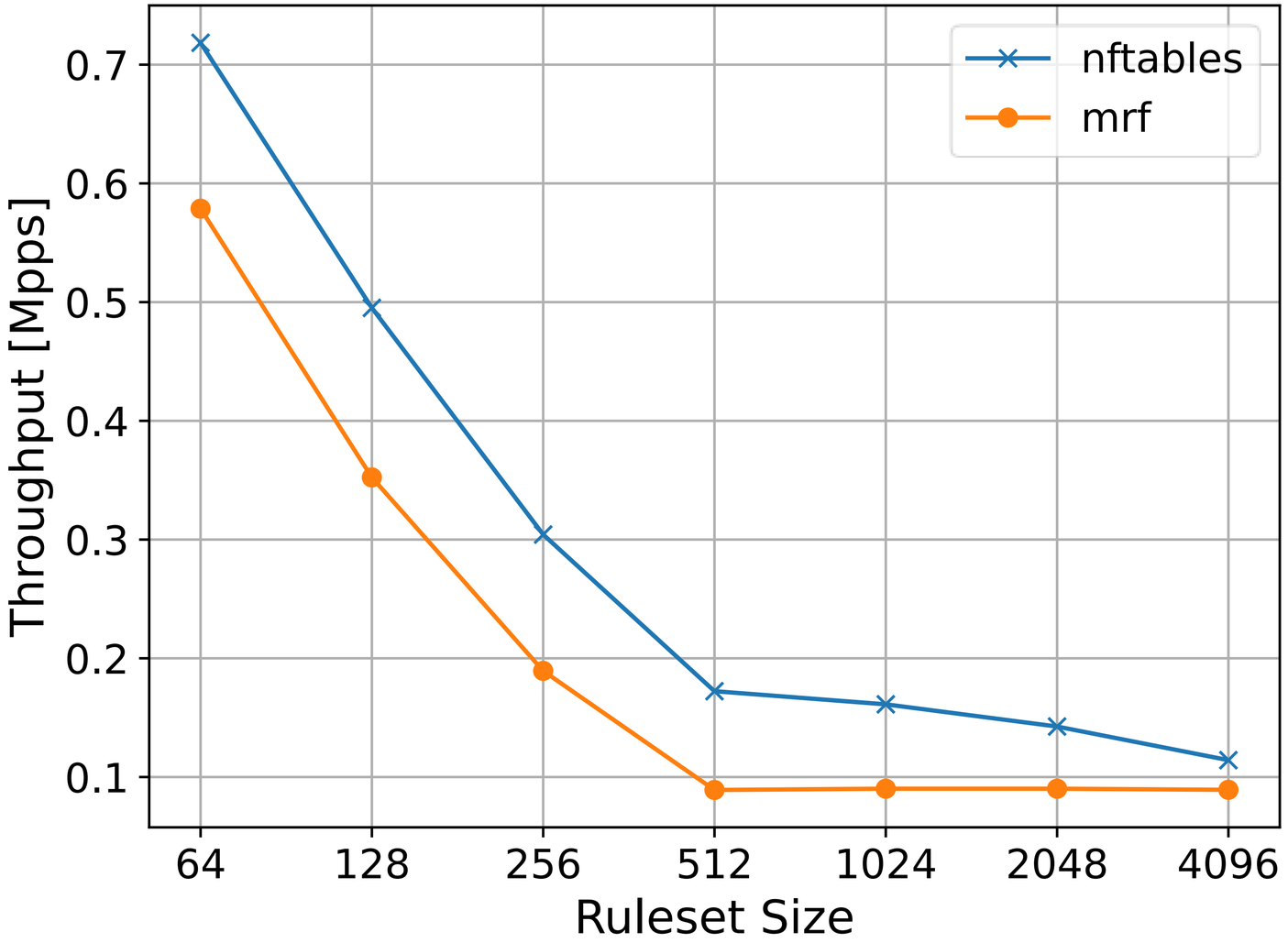
    p1 → p2 → … → pX → repeat



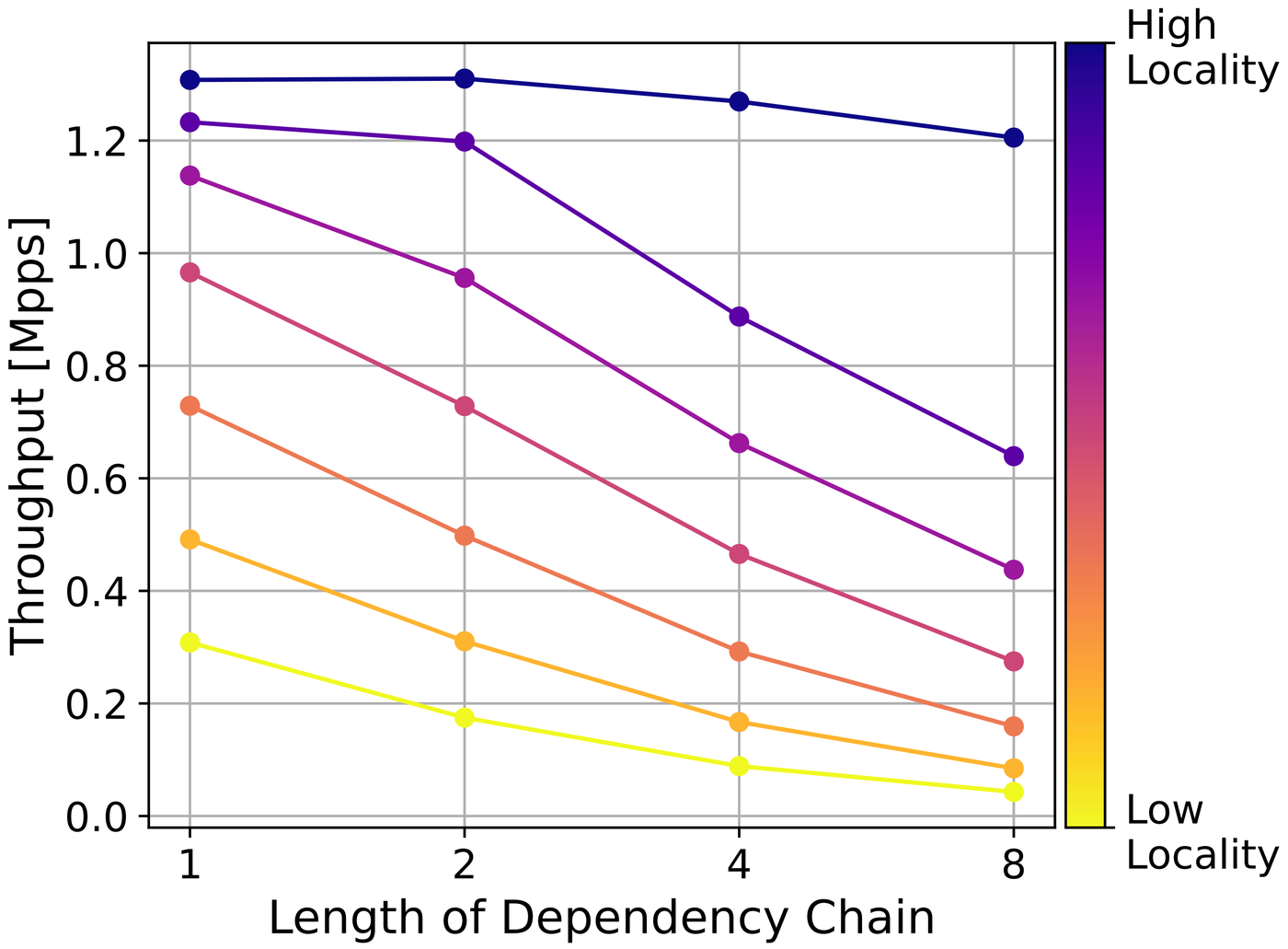
Evaluation: Single Core, High Locality



Evaluation: Single Core, Worst Case

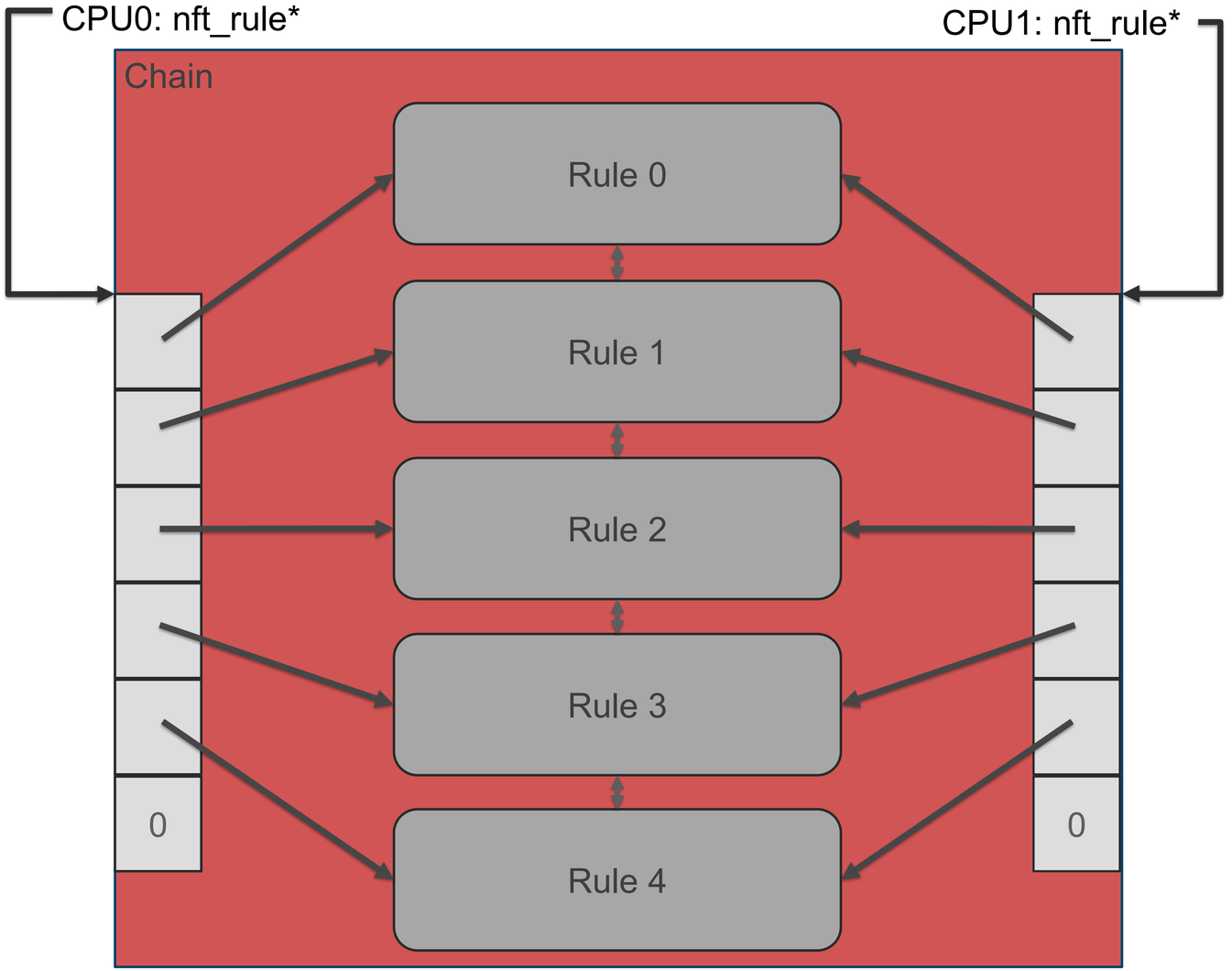


Evaluation: Single Core

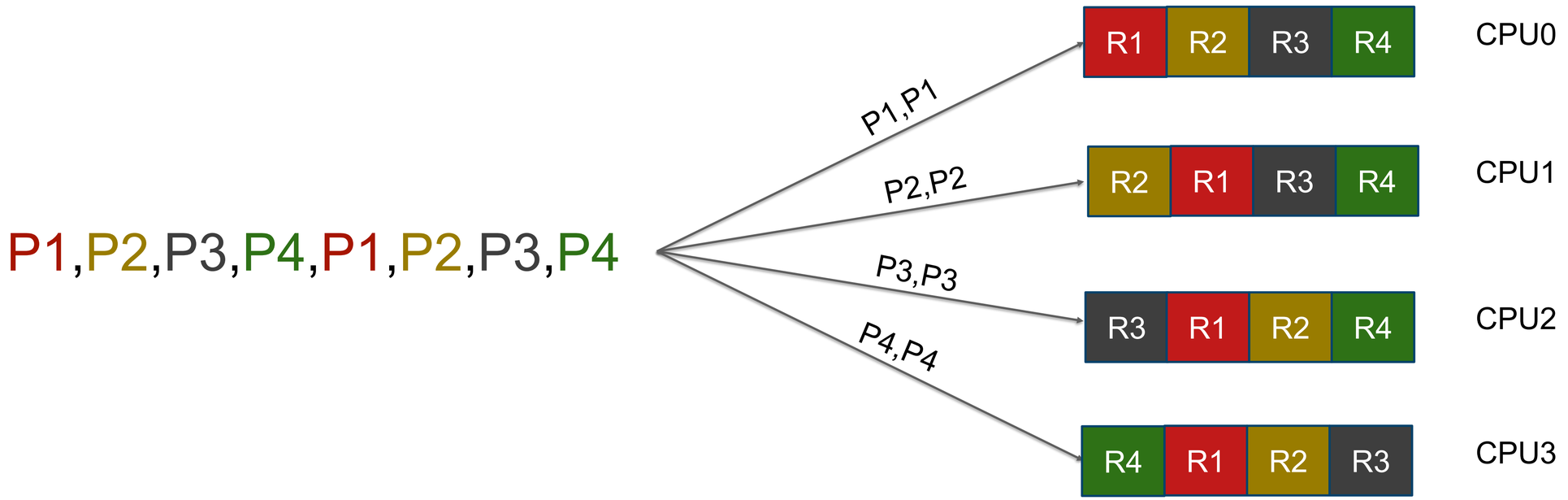


Scaling Across Cores

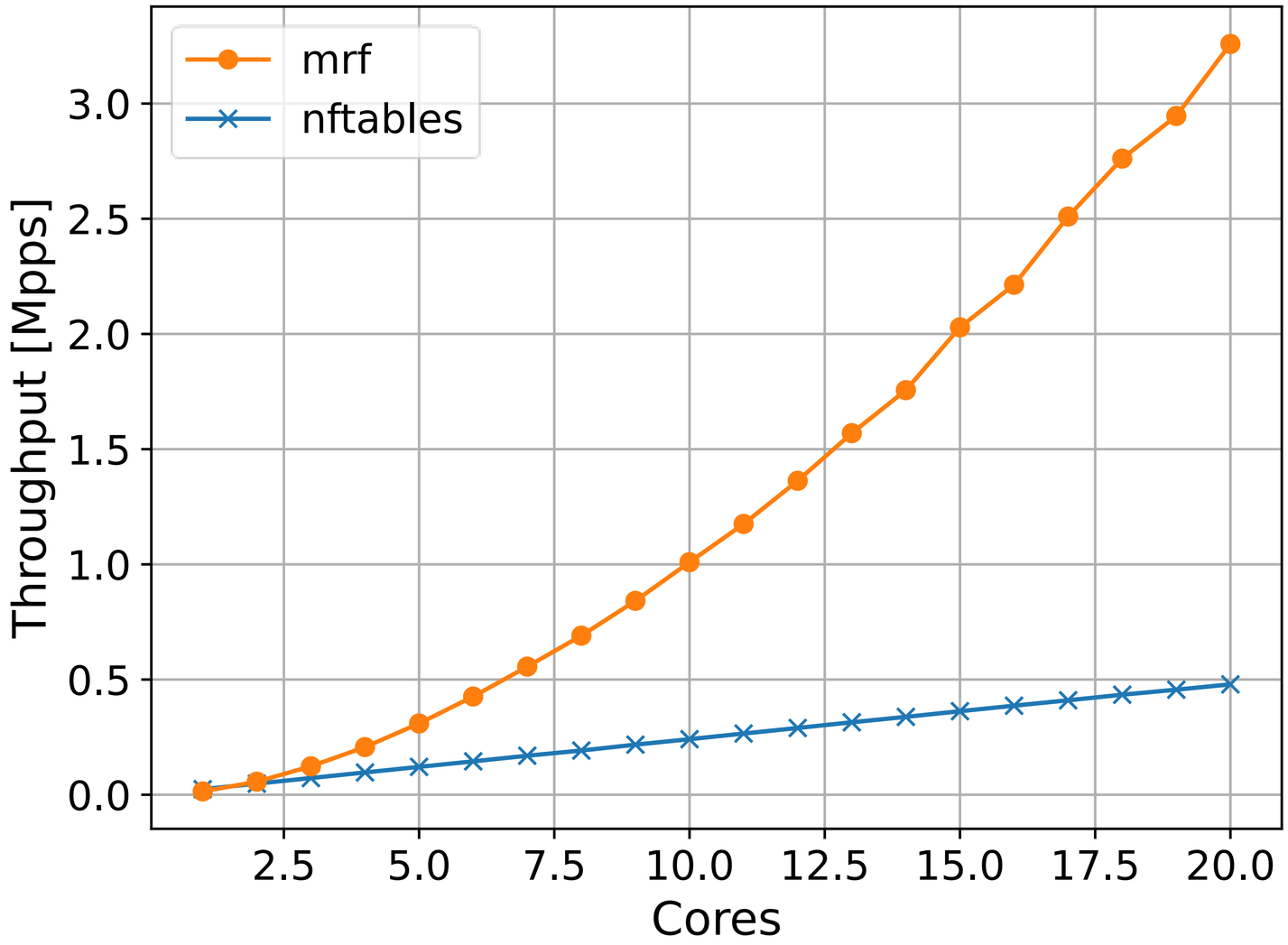
- Per-CPU pointer array
- Independent reordering per CPU
- Spread traffic across cores
  - Increase locality per core



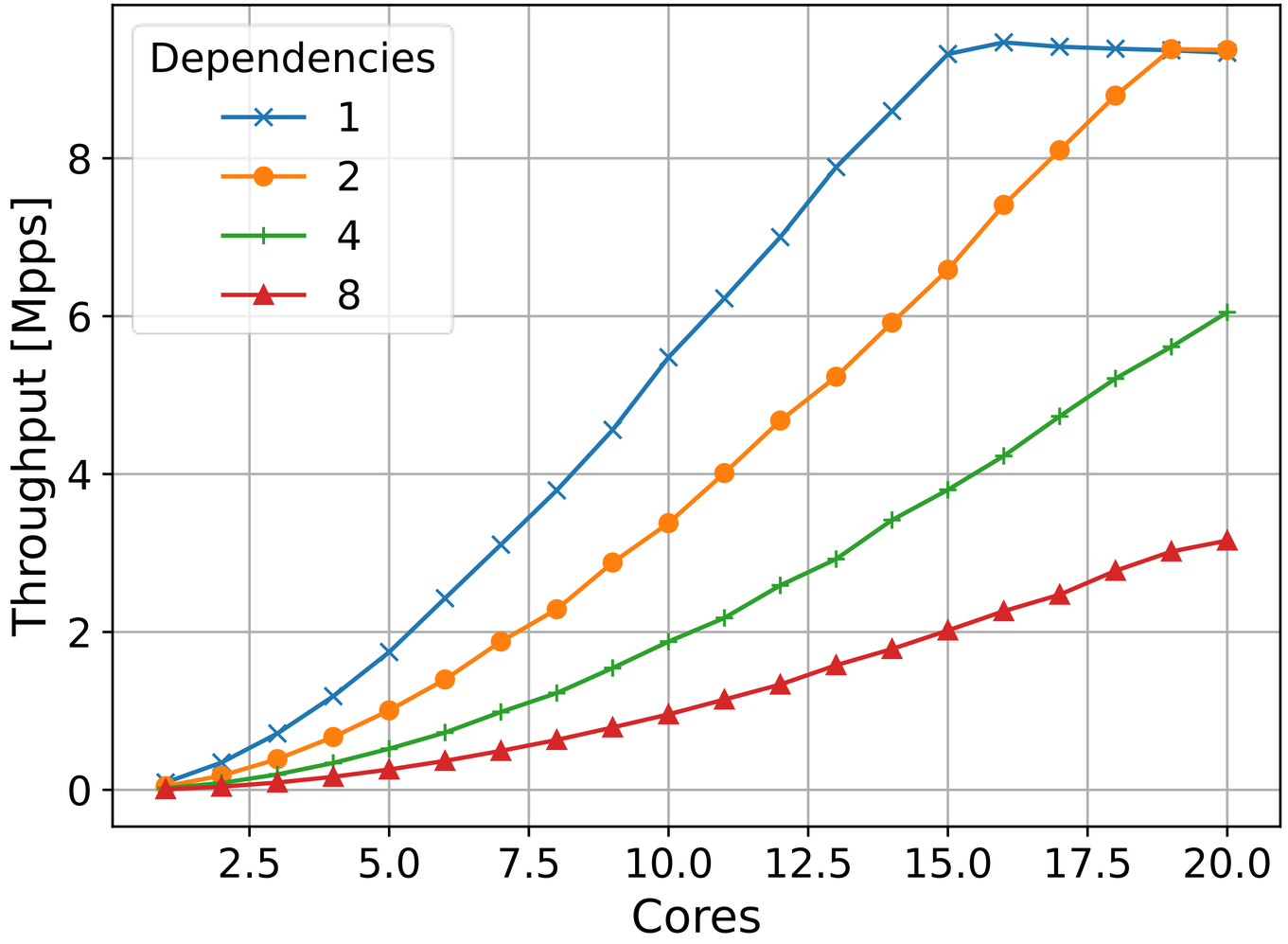
Scaling Across Cores



Evaluation: Multi-Core (4096 Flows)



Evaluation: Multi-Core (512 Flows)



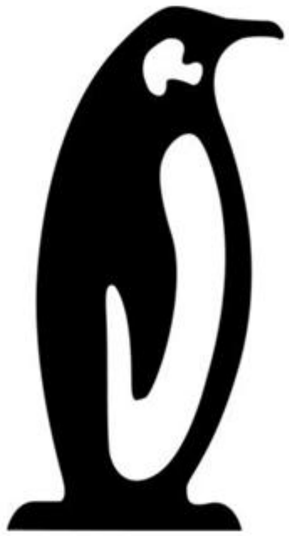
Conclusion

- A potential use case for MRF
- Opportunities and limitations of self-adjusting data structures
- Scaling behavior with locality-boosting load balancing
- Can we apply this in other domains?

Conclusion

- A potential use case for MRF
- Opportunities and limitations of self-adjusting data structures
- Scaling behavior with locality-boosting load balancing
- Can we apply this in other domains?

Questions?



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

